ITPROFEL2: OPEN SOURCE PROGRAMMING

Learning Module: Finals - Week 3

Instructor: Henry V. Dela Cruz

Topic: Functions and Modular Programming

I. Overview and Objectives

As programs grow in size, writing all logic in a single top-to-bottom file becomes impossible to maintain. This week, we transition into modular programming by learning how to encapsulate logic into reusable blocks called functions.

Intended Learning Outcomes (ILOs):

- Define functions and explain their importance in modular software development.
- Differentiate between Built-in functions and User-Defined (User-Friendly) functions.
- Write, define, and call custom functions with parameters and return values.
- Analyze the Concept of Scope (Local vs. Global variables).
- Demonstrate how to assign a function to a variable as a first-class object.

II. What is a Function?

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing. This aligns perfectly with the open-source philosophy of writing tools that others can easily plug into their own projects.

Built-in vs. User-Defined Functions

Python provides many **built-in functions** like `print()`, `len()`, and `type()`. These are available out of the box. However, we can also create our own **user-defined functions** (sometimes referred to as user-friendly functions) to handle logic specific to our application's needs.

III. Defining and Calling Functions

In Python, you define a function using the `def` keyword, followed by the function name, parentheses `()`, and a colon `:`. The indented block of code that follows is the function body.

```
# 1. Defining the function
def greet_user(username):
    # The 'return' statement sends a result back to whoever called the function
    return f"Welcome to the dashboard, {username}!"

# 2. Calling the function
# We must pass the required argument ("Henry") into the parentheses
message = greet_user("Henry")
print(message)  # Output: Welcome to the dashboard, Henry!
```

IV. The Concept of Scope

Scope refers to the visibility of variables. Variables defined *inside* a function body have a **Local Scope**, and those defined *outside* have a **Global Scope**.

- A local variable can only be accessed within the function it was created in.
- If you try to access a local variable from the global space, Python will throw a `NameError`.

```
global_var = "I am accessible everywhere."

def test_scope():
    local_var = "I only exist inside this function."
    print(global_var)  # This works
    print(local_var)   # This works

test_scope()
print(global_var)      # This works
# print(local_var)     # ERROR! local_var is not defined in the global scope.
```

V. Assigning a Function to a Variable

In Python, functions are "first-class objects." This means you can treat functions just like any other variable. You can assign a function to a new variable name, pass it as an argument, or return it from another function.

```
def shout(text):
    return text.upper()

# Assign the function 'shout' to a new variable called 'yell'
# Note: We do NOT use parentheses here, because we are assigning the function object itself, not call
yell = shout

# Now 'yell' acts exactly like 'shout'
print(yell("hello open source!"))  # Output: HELLO OPEN SOURCE!
```

Laboratory Coding Session: Data Sanitization Module

Scenario: You are tasked with creating a data sanitization utility. User inputs (like names) often come in with inconsistent capitalization and extra spaces. You need to write modular, reusable functions that clean this data and generate a standardized system format.

Instructions (Your code must implement these exact rules):

1. Define a function named `clean_name(raw_name)`.
   - This function must take a string, remove any leading/trailing whitespace using `.strip()`, format it using `.title()`, and **return** the result.
2. Define a second function named `generate_system_id(clean_name)`.
   - This function takes the cleaned name, removes all spaces using `.replace(" ", "")`, converts it entirely to lowercase using `.lower()`, appends the string `"_sys"` to the end, and **returns** the result.
3. In the global scope of your script, ask the user to `input()` a messy name (e.g., `" jUaN deLa CRuz "`).
4. Call your functions sequentially to process the input and print the final results.
5. *Bonus:* Assign your `clean_name` function to a variable named `sanitizer` and use that variable to make the call.

Sample Program Run (Expected Output):

```
--- DATA SANITIZATION UTILITY ---
Enter a messy name string:    hEnRy deLa cRuz

Processing...

Cleaned Name: Henry Dela Cruz
Generated System ID: henrydelacruz_sys
```

Submit your completed `.py` *file to the instructor. Keep these functions safe, as reusable modules like this will be essential soon.*